

8 / 10 件 最終実行: 2026-03-25 19:41

High Medium Low All

HIGH zenn.dev 2026-03-25 17:54

『実践Claude Code入門』を読みながらDev Container環境でPlaywright MCPを動かしてみた

- 書籍『実践Claude Code入門』を参照し、Dev Container環境でPlaywright MCPを動かした実践記録。
- AIエージェントがブラウザのインストール、依存パッケージの不足、日本語フォントの文字化けなどを自律的に解決する過程を詳述。
- Webページのスクリーンショット撮影や要素操作を通じて、簡易的なE2Eテストへの活用可能性が示された。

▼ 判定理由

今読む理由: AIエージェントとMCPの連携による開発自動化の実践例と、Dev Container環境での具体的なトラブルシューティングが詳細に記録されており、同様の技術を試す際に非常に参考になる。

後回し理由: Claude CodeやPlaywright MCP、Dev Containerといった特定の技術要素に関心が薄い場合は、すぐに読む必要性は低いかもしれない。

Claude Code Playwright MCP Dev Container AIエージェント ブラウザ自動化

あとで読む（読まない）を何とかしたい



星影 | Tech Hunter

2026年3月26日 02:00

Raindropに、あとで読みたい記事を溜めている。

星影

@unsoluble_sugar · フォローする

X

返信先: @unsoluble_sugarさん

Pocketの代わりはRaindrop使うことにしました🌧️
noteのニュースめもで取り上げるネタとか、気にな
った記事とりあえずここに全部ブチ込む感じで😊
[note.com/unsoluble_suga...](https://note.com/unsoluble_sugar/n/n543659f80c1e)

午後11:06 · 2025年6月3日 ⓘ

♥️

💬 返信

🔗 リンクをコピー

Xでもっと読む

その場ではちゃんと「これは読んでおきたい」と思って保存しているはずなのに、あとで見返すと、読まないまま埋もれている記事がかなり多い。

あとで読む、というより「あとで読まない」の保管庫になりつつある。

短いニュースなら、まだ回る。日々のニュースチェックはnoteで「ニュースめも」としてアウトプットしており、気になった話題をざっと読み、要点を拾って、ひとこと添えるくらいなら習慣として回している。

その日見かけて気になったニュースをひたすら記録  キュレーション

メディア的にご活用ください〜   一部無料公開  

note.com

この運用はわりと自分に合っていると思う。

だが、長文記事になると話が変わる。インタビュー記事や考察の厚い記事、海外の長文エントリ。そういうものほど「ちゃんと読みたい」という気持ちが先に立つ。

その結果、まとまった時間が取れる時に読もうと思って後回しにし、そのまま読まれずに沈んでいく。

短いニュースは、流れの中で処理できる。でも長文記事は「腰を据えて向き合うもの」として扱ってしまう。その丁寧さが、かえって読むハードルを上げているのかもしれない。

あとで読むが溜まっていくと、情報のストックというより、未処理タスクの山に見えてくる。読む気があるから保存したはずなのに、一覧に並んでいるだけで気が重くなる。

これでは未読リストではなく、判断を先送りした履歴に近い。

たぶん必要なのは、気合いではない。

仕組みなのだと思う。

実際、日々のIT系ニュースについては、毎日定期収集して一覧化する仕組みをすでに自分で作って運用している。

今日のテックニュース (2026-03-25)

忙しいエンジニアのために。毎日サクッとキャッチアップ。

unsolublesugar.github.io

だったら、Raindropの「あとで読む」も似たように扱えるのではないか。

たとえば、一定量溜まったら、あるいは数日おきに一度、特定のコレクションを走査する。そこに入っている記事のタイトルや本文を拾い、AIで短い要約をつけて並べる。

それだけでも、未読の山はかなり見え方が変わるはずだ。

いまの「あとで読む」が辛いのは、記事の中身が見えないまま重なっていくからだ。タイトルだけ見えている長文記事の束は、それだけで重い。

でも、1本ごとに短い要約がついていれば、「これは今読むべき」「これは後回しでいい」「これはもう読まなくてもよさそう」が判断しやすくなる。

要するに、全文を読む前に、AIに入口だけ先に作ってもらう。読むか読まないかの判断を、毎回ゼロからやらなくて済むようにする。

本当に欲しいのは、保存した記事を全部完読することではなく、未読の山をさばける状態にすることなのだと思う。保存した記事をすべて読むのは、たぶん無理だ。

でも、読まないまま積み上げるしかない状態は変えられるかもしれない。

「あとで読む」を気合いの問題として抱えるのではなく、読み始めるまでの摩擦を減らす仕組みに変える。まず何とかしたいのは、記事そのものではなく、その手前にある重さなのだ。

...と、ここまで考えて、結局また「考えて終わる」だけになるのも違うなと思い、仕様の大枠をChatGPTに相談し、実装はClaude Codeに手伝ってもらいながら形にしてみることにした。

Raindrop.io の特定コレクションを定期取得し、保存記事の要約一覧を生成する仕組みを設計したいです。実装前提で、作業計画と仕様を具体化してください。

背景

自分は Raindrop に「あとで読む」記事を溜めています
が、長文記事やボリュームのある記事ほど後回しになり、
読まずに埋もれていくことが多いです。

短いニュースは日々チェックして note に「ニュースめも」
としてアウトプットできていますが、調査記事、インタビュー、
考察系、海外の長文記事などは、まとまった時間が
必要だと思っしまい、結果として未読の山になりがちです。

https://note.com/unsoluble_sugar/m/m66f6d8989a3e

この課題に対して、気合いで読むのではなく、AI を使って
「読む前の前処理」を自動化したいです。

具体的には、Raindrop の特定コレクションに溜まった記事
を一定間隔で走査し、各記事の内容を要約して一覧化する
ことで、

- 今読むべきか
 - 後回しでよいか
 - 読まずに捨ててよいか
- を判断しやすくしたいです。

既存の前提

思考時間: 36s >

以下の方針が、いちばん現実的です。

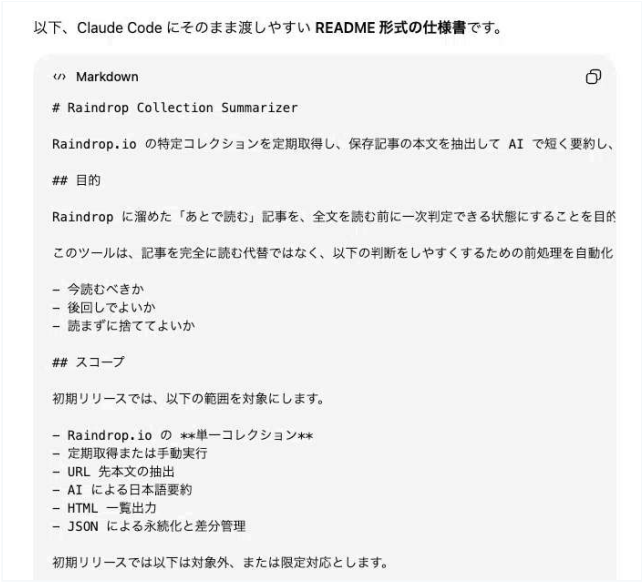
Raindrop の特定コレクションを定期取得 → URL先本文を抽出 → AIで短く要約・判定補助 → JSONを正本に保存 → Markdown/HTMLを派生生成 → GitHub Pages で一覧確認、
という流れです。Raindrop API ではコレクション一覧取得、コレクション単位の
raindrop 取得、ページング、perpage 最大 50、sort=-created、nested=true などが
使えます。テスト用途なら自分用の Test token も利用できます。GitHub Actions は cron
スケジュール実行と secrets 管理に対応しているので、自動運用とも相性がいいです。本
文抽出は Python なら Trafilatura を第一候補にするのが扱いやすいです。

`developer.raindro...` +7

1. この仕組みの目的と前提の整理

目的

この仕組みの目的は、記事を「読む」ことそのものではなく、未読スタックを意思決定し
やすい単位に圧縮することです。



そうして出来たのが、Raindropの特定コレクションを定期的に拾い、本文を要約し、優先度つきで並べる小さなダッシュボードだ。



Tsuyu-mi

Raindrop → 雫（しずく）→ 露（つゆ）→ 露見（つゆみ） — 溜まった記事の中身を「露わに見る」という意味を込め「Tsuyu-mi」と名付けている。

意味のある名前を付けると、愛着も湧く。

コストはミニマムで抑えたかったので、RaindropのAPIとGoogle Gemini（gemini-2.5-flash）を採用した。

🔗 3. API キーの取得

このツールの実行には **Raindrop.io API トークン** と **LLM API キー** の 2 つが必要です。

🔗 Raindrop.io テストトークン

1. [Raindrop.io Integrations](#) にアクセス
2. 「For Developers」セクションの **Create new app** をクリック
3. アプリ名を入力（例: `RaindropSummarizer`）して作成
4. 作成したアプリをクリック → **Create test token** をクリック
5. 表示されたトークンをコピー

🔗 コレクション ID の確認

1. [Raindrop.io](#) にアクセス
2. 対象のコレクション（「あとで読む」等）を開く
3. URL を確認:
`https://app.raindrop.io/my/{collection_id}` — この数値部分がコレクション ID

🔗 LLM API キー

以下のいずれか 1 つのプロバイダーの API キーを取得します。

Google Gemini（推奨・無料枠あり）

1. [Google AI Studio](#) にアクセス
2. **Create API Key** → **Create API key in new project** を選択
3. 推奨モデル: `gemini-2.5-flash`

💡 Tip

Google Cloud の請求先アカウントの紐づけと [Gemini API の有効化](#) が必要な場合があります。

OpenAI

1. [OpenAI API Keys](#) にアクセス
2. **Create new secret key** でキーを生成
3. 推奨モデル: `gpt-4.1-mini`

Anthropic

1. [Anthropic Console](#) にアクセス
2. **Create Key** でキーを生成
3. 推奨モデル: `claude-haiku-4-5-20251001`

こんな感じで「あとで読む」を気合いで消化するのではなく、読む前の摩擦を減らす仕組みを作れば「負の連鎖が断ち切れるのでは？」という算段だ。

MEDIUM

zenn.dev 2026-03-10 23:04

予測市場の構造——集合知・市場設計・規制で理解する未来予測

- 予測市場の定義から、PolymarketやKalshiなどの具体例を通じて、そのメカニズムと背景を解説するZennの書籍の紹介です。
- 未来予測における市場の役割、歴史的経緯、基本メカニズム、市場設計、技術構造を体系的に学べます。
- 規制動向、インサイダー問題、主要プレイヤー比較、ユースケース、未来展望まで広範なトピックを網羅しています。

▼ 判定理由

今読む理由: 予測市場の全体像や詳細な構造、課題、未来について網羅的に学びたい場合に、本書籍の目次から全体像を把握するのに役立ちます。

後回し理由: 本書籍は多岐にわたる内容を網羅しているため、深い理解を得るにはまとまった読書時間が必要となります。

予測市場

集合知

市場設計

規制

Polymarket

実際、見え方を少し変えるだけで「読んでみよう」という気になるのだから、人間の心理は不思議なものだ。

とりあえず「エイヤ！」でバンプコーディングしてリリースしたので、やる気があれば気ままにアップデートしていく予定。リポジトリも公開してるので、興味があれば覗いてみてください。



unsolublesugar/tsuyu-mi: Fetch, summarize, and publish Raindrop.io articles as a priority-ranked HTML...

Fetch, summarize, and publish Raindrop.io articles as a priority-ranked HTML dashboard - unsolublesugar/tsuyu-mi

 GitHub

追記：開発の詳細についてZennに書きました。

「あとで読む（読まない）」を仕組みで解決する — AI 駆動の個人開発記

zenn.dev

以上「あとで読む（読まない）を何とかしたい」という話でした。

そんじゃーね